

Inverted Index Architecture and Vector Space Retrieval

Diego Besada Rodríguez

[\[My code\]](#) [\[Index\]](#) [\[Template\]](#) [\[Speech\]](#)

September 2026

1 Objectives

Scanning an entire collection of books every time we want to find a word is inefficient. This organizing, searching, and retrieving task across large text collections is known as Information Retrieval (IR). To tackle it, the notebook builds a pipeline around three main components: an inverted index to track where words appear, TF-IDF weighting to score how characteristic words are, and cosine similarity to measure how similar the books are to one another.

2 Method and Program Structure

2.1 Positional Inverted Index & Concordances

Rather than inspecting files linearly, the inverted index maps each unique word to the documents and positions where it appears. In Python, this is structured as a two-level nested dictionary:

```
master_idx = {'samlar': {'troll.txt': [641880, 654233], 'nils.txt': [51805, 118943]}}
```

Tokens are extracted with the Unicode regex `\p{L}+`, lowercased, and registered alongside their character start offsets (`w.start()`).

Boundary-Safe Concordance Extraction: Using stored character offsets, concordances are extracted with a window parameter W :

```
start = max(0, pos - W)
end = min(len(text), pos + len(word) + W)
concordance_text = text[start:end]
```

2.2 Document Representation via TF-IDF

Documents are projected into a Vector Space Model using Term Frequency-Inverse Document Frequency (tf-idf). The term frequency is normalized by the total token count of the document, and the idf dampens terms distributed broadly across the collection:

$$\text{tf}_{t,d} = \frac{\text{count}_{t,d}}{\text{total_words}_d}, \quad \text{idf}_t = \log_{10} \left(\frac{N}{\text{df}_t} \right), \quad w_{t,d} = \text{tf}_{t,d} \cdot \text{idf}_t \quad (1)$$

where $\text{count}_{t,d}$ is the number of times word t appears in document d , df_t is the number of documents containing t , and $N = 9$.

Because $N = 9$, idf_t ranges between 0 and $\log_{10}(9) \approx 0.954$. Words appearing across all 9 novels yield $\text{idf} = \log_{10}(1) = 0.0$, neutralizing common words without requiring a stop-word list.

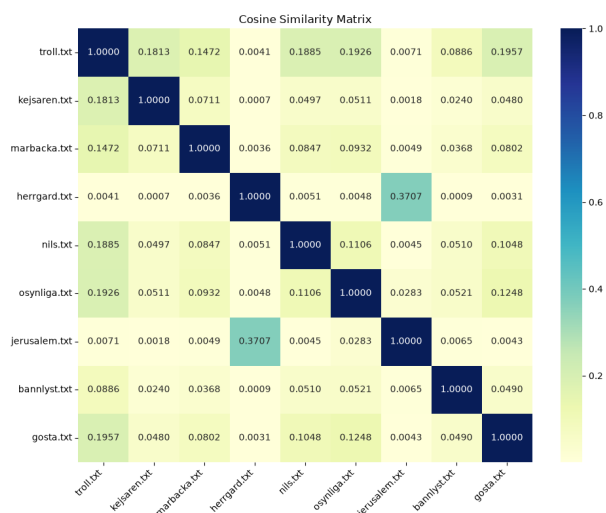
2.3 Cosine Similarity

To compare two documents, measuring the raw Euclidean distance or a simple dot product is misleading: longer books naturally accumulate higher word counts, making them appear distant from shorter texts even when sharing the exact same theme. Cosine similarity solves this by measuring the angle between document vectors rather than their magnitude, effectively normalizing for document length:

$$\text{sim}(d_1, d_2) = \frac{\sum_{t \in d_1 \cap d_2} w_{t,d_1} \cdot w_{t,d_2}}{\sqrt{\sum_{t \in d_1} w_{t,d_1}^2} \cdot \sqrt{\sum_{t \in d_2} w_{t,d_2}^2}} \quad (2)$$

The score ranges from 0 (no shared vocabulary) to 1 (identical relative word distribution). In the implementation, the generator `for word in doc1 if word in doc2` restricts the numerator to shared terms, skipping the vast majority of inactive vocabulary dimensions without altering the result.

3 Results and Conclusion



By computing the cosine similarity pairwise between all document vectors, we construct the 9×9 matrix. The matrix is symmetric because the cosine similarity between novel a and novel b is identical to that between b and a , and the diagonal is naturally 1.0000 since every novel is compared against itself.

Across the collection, the similarity scores are generally quite low (mostly under 0.20), showing that the novels do not share an extensive common vocabulary. The only clear exception is the pair `herrgard.txt` and `jerusalem.txt`, which stands out noticeably from the rest with a similarity of 0.3707.

Overall, the inverted index and vector space model provide a clear and straightforward way to compare documents without complex models. However, two limitations are worth noting: tokenizing surface words without lemmatization treats different grammatical forms of the same root as unrelated dimensions, and extracting concordances using fixed character offsets instead of a word-based window frequently cuts words in half at the boundaries.

4 Further Questions

4.1 Relation to Jeff Dean's Index Encoding

Slide 45 describes Google's original index structure (1997–1999):

WORD -> [docid + nhits] [hit] [hit] ... [docid + nhits] [hit] ...

Each entry maps a word to a document ID along with an explicit occurrence counter (**nhits**), followed by the exact positions and formatting attributes for each occurrence (**hit**).

Our implementation follows the same conceptual hierarchy, with distinct engineering trade-offs:

- **Occurrence Count:** Google explicitly stored **nhits** as an integer in the document header so the binary reader knew how many 16-bit hits to consume next. In our nested dictionary, each document entry directly contains the list of word positions, making an explicit counter redundant since the length of this list directly provides **nhits** for our tf calculations.
- **Positional Hits:** Both systems store exact word positions rather than just boolean presence, allowing proximity queries, concordance extraction, and contextual snippets.
- **Storage vs. High-Level Abstraction:** Google packed these values into tightly aligned binary integers (32-bit and 16-bit) to minimize disk footprint and memory bandwidth across billions of web pages. In contrast, our pipeline relies on high-level Python objects serialized to human-readable JSON.

4.2 Steve Jobs's Speech at Lund University (1985)

In his 1985 address at Lund University, Steve Jobs reflected on the static nature of books, noting that while printed text preserves a thinker's ideas, it cannot sustain an interactive dialogue or answer questions. He projected that computers would eventually amass human knowledge into dynamic systems capable of two-way interaction. Our laboratory project mirrors the fundamentals of this vision: by indexing Selma Lagerlöf's novels with character offsets and tf-idf weighting, we convert static, linear books into a structured database. Tools like concordance lookup and vector comparisons replace passive reading with targeted queries, laying the algorithmic groundwork for modern interactive search and Retrieval-Augmented Generation (RAG).